

AI/ML intern DAY 3

Task 1 - Human Pose Detection

LOYA PRANAY (pranay8679@gmail.com)

Date: 13-05-2026

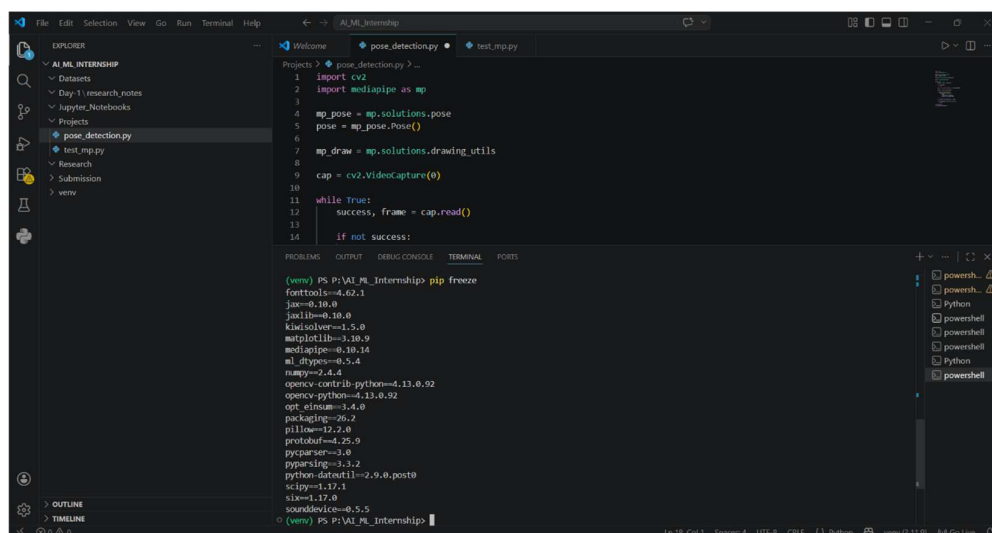
1. Objective

The objective of this task was to implement a real-time human pose detection system using MediaPipe and OpenCV. The system was designed to detect body landmarks and display skeleton tracking dynamically using webcam input.

The task also helped in understanding computer vision workflows, body landmark detection, and real-time human movement tracking using AI technologies.

2. Technologies Used

Technology	Purpose
Python	Programming language
OpenCV	Webcam access and image processing
MediaPipe	Human pose landmark detection
VS Code	Development environment



```
1 import cv2
2 import mediapipe as mp
3
4 mp_pose = mp.solutions.pose
5 pose = mp_pose.Pose()
6
7 mp_draw = mp.solutions.drawing_utils
8
9 cap = cv2.VideoCapture(0)
10
11 while True:
12     success, frame = cap.read()
13     if not success:
14         break
15
16     pose.process(frame)
17     mp_draw.draw_landmarks(frame, pose.pose_landmarks, mp_pose.POSE_CONNECTIONS)
18     cv2.imshow('Human Pose Detection', frame)
19     if cv2.waitKey(1) & 0xFF == ord('q'):
20         break
21
22 cap.release()
23 cv2.destroyAllWindows()
```

(view) PS P:\AI_ML_Internship> pip freeze

fonttools==4.62.1
jax==0.10.0
jaxlib==0.10.0
keras==3.0.0
matplotlib==3.10.0
mediapipe==0.10.16
ml-dtypes==0.5.4
numpy==2.4.4
opencv-contrib-python==4.13.0.92
opencv-python==4.13.0.92
opt_einsum==3.4.0
packaging==26.2
pillow==12.2.0
protobuf==4.25.5
pycparser==3.0
pyparsing==3.3.2
python-dateutil==2.9.0.post0
scipy==1.17.1
six==1.17.0
sounddevice==0.5.5

3. Understanding Human Pose Detection

Introduction

Human pose detection is a computer vision and Artificial Intelligence (AI) technique used to detect and track body landmarks from images or videos.

The system identifies important body points such as:

- shoulders
- elbows
- wrists
- hips
- knees
- ankles

These landmarks are connected together to form a digital body skeleton.

Pose detection allows AI systems to understand:

- posture
- movement
- gestures
- body orientation

Applications of Pose Detection

Pose detection is used in:

- fitness tracking
- yoga posture correction
- sports analysis
- gaming
- healthcare monitoring
- virtual try-on systems
- motion tracking

4. Workflow of the Pose Detection System

The pose detection workflow includes the following steps:

Step 1 — Webcam Access

OpenCV accesses the webcam and captures live video frames.

Step 2 — Image Conversion

Frames are converted from BGR format to RGB format because MediaPipe processes RGB images.

Step 3 — Pose Detection

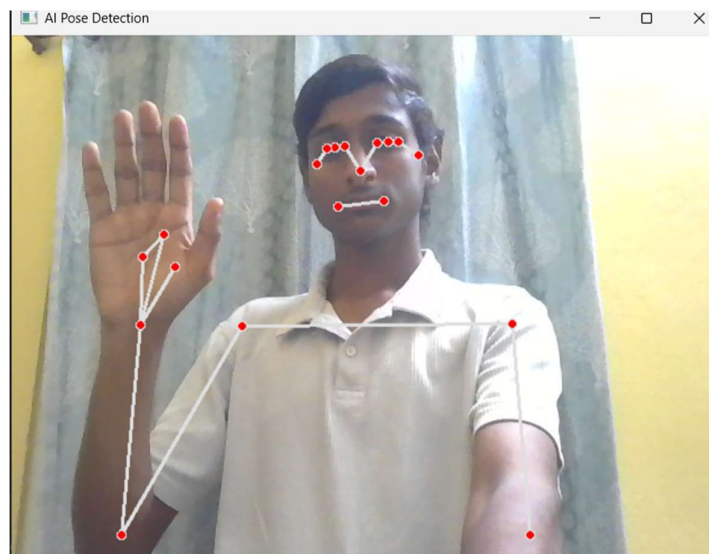
MediaPipe detects body landmarks using AI-based pose estimation models.

Step 4 — Skeleton Tracking

Detected landmarks are connected to form a body skeleton structure.

Step 5 — Real-Time Output

The webcam displays real-time body tracking and movement analysis.



5. Code Implementation

Pose Detection Program

Paste your working code here:

```
import cv2
import mediapipe as mp

mp_pose = mp.solutions.pose
pose = mp_pose.Pose()

mp_draw = mp.solutions.drawing_utils

cap = cv2.VideoCapture(0)

while True:
    success, frame = cap.read()

    if not success:
        break

    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    results = pose.process(rgb)

    if results.pose_landmarks:
        mp_draw.draw_landmarks(
            frame,
            results.pose_landmarks,
            mp_pose.POSE_CONNECTIONS
        )

    cv2.imshow("AI Pose Detection", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

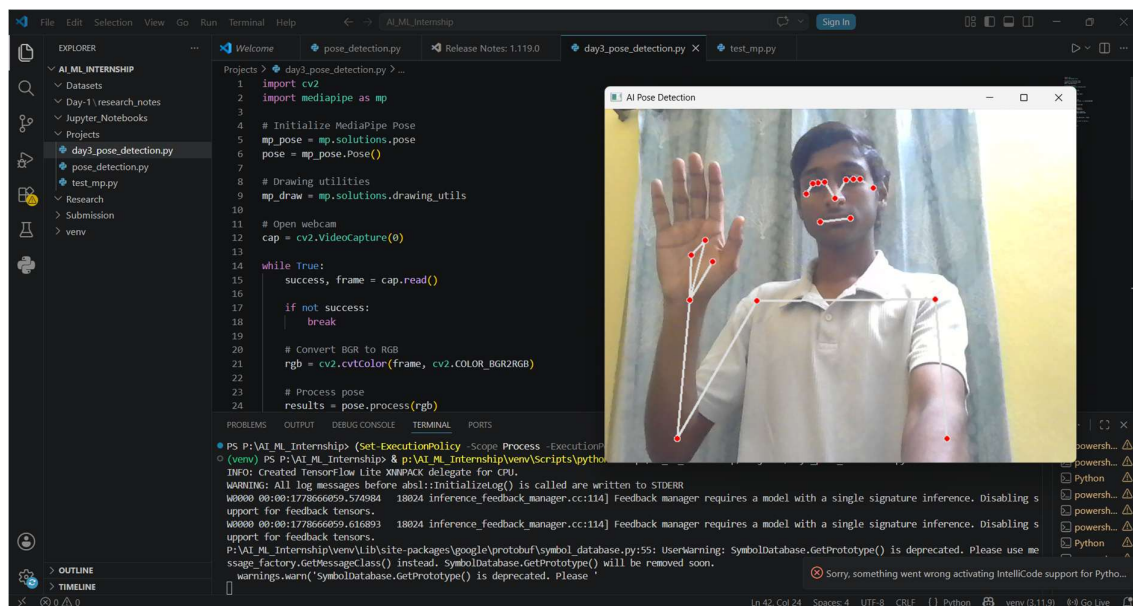
6. Output and Results

The pose detection system successfully:

- accessed the webcam
- detected body landmarks
- displayed body skeleton tracking
- tracked body movement dynamically in real time

The implementation demonstrated the practical use of:

- Artificial Intelligence
- Computer Vision
- Pose Estimation
- Real-Time Video Processing



7. Challenges Faced and Solutions

Challenge Faced	Solution Applied
MediaPipe compatibility issues	Installed compatible package versions
OpenCV import errors	Installed OpenCV inside virtual environment
Virtual environment confusion	Activated venv correctly
Storage issues in C drive	Shifted workspace to P drive
Webcam access issues	Enabled camera permissions

8. Learning Outcomes

During this task, the following concepts and skills were learned:

- AI environment setup
- Virtual environment management
- OpenCV basics
- MediaPipe pose detection
- Human body landmarks
- Real-time webcam processing
- Skeleton tracking systems
- Computer vision workflows
- Debugging AI environments

9. Conclusion

This task provided practical exposure to real-time human pose detection using AI and computer vision technologies.

A working pose detection system was successfully implemented using MediaPipe and OpenCV. The project improved understanding of body landmark detection, skeleton tracking, and real-time movement analysis.

The task also helped strengthen practical skills in Python programming, computer vision workflows, and AI-based body tracking systems.